COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	8/28.26	**DATE SUBMITTED:**	
NAME:	Rayven Ferry Licas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 SEARCHING TECHNIQUES

1. PROCEDURE OUTPUT

What procedures did you perform?

5. Procedure

ILO A: Create C++ code for searching data using different techniques

We will implement our searching technique on a data filled with random values. To begin, we want the following directives:

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
```

Then, we will define the capacity of data elements in our dataset.

```
const int max_size = 50;
```

This will be used to define your dataset. Your driver code must have the following:

```
//generate random values
int dataset[max_size];
srand(time(0));
for(int i = 0; i < max_size; i++){
    dataset[i] = rand();
}
//show your datasets content
for(int i = 0; i < max_size; i++){
    std::cout << dataset[i] << " ";
}
```

Once you are sure that you can generate random integers for your dataset, you can delete the second iterative loop and move on with the creation of the different searching techniques. **Verify this by including a screenshot of the generated data upon successful implementation of the code above in table 6-1.**

This implementation will also require the use of linked lists, so create a file "nodes.h" in your C++ project which will contain the definition of our single linked list node:

```
template <typename T>
class Node{
public:
    T data;
    Node *next;
};
```

```
        //linear search
        linearLS(name1, 'n');
```

The sequential search algorithm does not change even if it operates on a linked list. **Show your mo from earlier so that it will operate on a linked list. Screenshot your code and output with corresp observations/notes in table6-2b.**

B.1 Binary Search

This algorithm must also be included in your "searching.h" file.

```
BINARY SEARCH ALGORITHM
Step 1: low i=0, up=n-1
Step 2: Repeat while low <= up
   mid = int(low+up)/2
  if(no=arr[mid])
    print "Search element is found!"
    exit
    else
  if(no < arr[mid]) then
     up =  mid-1
else
     low = mid+1
Step 3: Print "Search element is not found"
Step 4: Exit
```

Create a function called binarySearch that accepts necessary parameters to execute the algorith described above. Indicate your code, output, and observations in table 6-3a.

Next, just like in the linked list with linear search, we will perform binary search in a linked list. Hov are pre-requisites for this to work: the linked list must have ordered values and we must know hov nodes exist in your list. For our testing, we can create our linked list with the following code:

```
//create a linked list for binary search
//make sure the values are ordered
char choice = 'y'; int count = 1; int newData;
Node<int> *temp, *head, *node;

while(choice=='y'){
    std::cout << "Enter data: ";
    std::cin >> newData;

    if(count==1){
        head = new_node(newData);
        std::cout << "Successfully added " << head->data << " to the list.\
        count++;
    } else if(count==2) {
        node = new_node(newData);
        head->next = node;
        node->next = NULL;
        std::cout << "Successfully added " << node->data << " to the list.\
```

```
        count++;
    } else {
        temp = head;
        while(true){
            if(temp->next == NULL) break;
            temp = temp->next;
        }
        node = new_node(newData);
        temp->next = node;
        std::cout << "Successfully added " << node->data << " to the list.\n";
        count++;
    }

    //when to end
    std::cout << "Continue? (y/n)";
    std::cin >> choice;
    if(choice=='n') break;
}
```

An optional snippet of code can be added here to verify that our data is actually stored onto a linked list as intended.

```
//display
Node<int> *currNode;
currNode = head;
while(currNode!=NULL){
    std::cout << currNode->data << " ";
    currNode = currNode->next;
}
```

Binary search still works the same, however. We need to create a function that will enable us to find the middle node. This function, getMiddle, must take the first and last node as its parameters. It follows the algorithm described below:

1. Traverse the singly linked list using two pointers.
2. Move one pointer by one step ahead and the other pointer by two steps.
3. When the fast pointer reaches the end of the singly linked list, the slow pointer will reach the middle of the singly linked list.
4. Return slow pointer address.

We will use this function to find the middle node of our linked list to properly search through the sorted list. Next, another function to implement the binary search. Step 2 of this algorithm uses the function from the algorithm above.

1. Start node is set to head of the list and the last node is set to NULL.
2. Middle element is calculated using the two pointers approach discussed above.
3. If the middle element is same as the key to be searched, we return it.
4. Else if middle element is greater than the key to be searched, we have to search is the right side of the singly linked list. So, we set start pointer to the next of middle element.
5. Else if middle element is less than the key to be searched, we have to search is the left side of the singly linked list. So, we set last pointer to the middle element.
6. If the key is found or the entire linked list gets traversed, we break the loop.


```
        count++;
    } else {
        temp = head;
        while(true){
            if(temp->next == NULL) break;
            temp = temp->next;
        }
        node = new_node(newData);
        temp->next = node;
        std::cout << "Successfully added " << node->data << " to the list.\n";
        count++;
    }

    //when to end
    std::cout << "Continue? (y/n)";
    std::cin >> choice;
    if(choice=='n') break;
}
```

An optional snippet of code can be added here to verify that our data is actually stored onto a linked list as intended.

```
//display
Node<int> *currNode;
currNode = head;
while(currNode!=NULL){
    std::cout << currNode->data << " ";
    currNode = currNode->next;
}
```

Binary search still works the same, however. We need to create a function that will enable us to find the middle node. This function, getMiddle, must take the first and last node as its parameters. It follows the algorithm described below:

1. Traverse the singly linked list using two pointers.
2. Move one pointer by one step ahead and the other pointer by two steps.
3. When the fast pointer reaches the end of the singly linked list, the slow pointer will reach the middle of the singly linked list.
4. Return slow pointer address.

We will use this function to find the middle node of our linked list to properly search through the sorted list. Next, another function to implement the binary search. Step 2 of this algorithm uses the function from the algorithm above.

1. Start node is set to head of the list and the last node is set to NULL.
2. Middle element is calculated using the two pointers approach discussed above.
3. If the middle element is same as the key to be searched, we return it.
4. Else if middle element is greater than the key to be searched, we have to search is the right side of the singly linked list. So, we set start pointer to the next of middle element.
5. Else if middle element is less than the key to be searched, we have to search is the left side of the singly linked list. So, we set last pointer to the middle element.
6. If the key is found or the entire linked list gets traversed, we break the loop.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```cpp
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function

const int max_size = 50;

    //generate random values
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
    }
    //show your datasets content
    for(int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.